TECNOLÓGICA DEL ORIENTE
Institución de Educación Superior
Arquitectura de Software · Unidad 1: Principios de arquitectura de software

AulaNube

Arquitectura de microservicios en la nube para la plataforma de capacitación corporativa multiempresa de CapacitaPro S.A.S.

Informe técnico · Actividad «Orquestando Códigos: La Sinfonía de los Sistemas»

Integrantes
[Nombre completo] — líder del equipo
[Nombre completo]
[Nombre completo]
[Nombre completo]

Docente: [Nombre del docente]

Repositorio: https://github.com/augustocco/aulanube
Prototipo: https://aulanube.149-130-191-139.sslip.io

Bucaramanga, octubre de 2026

Contenido

1. Introducción 3
2. Caso de estudio: CapacitaPro S.A.S. 3
2.1 Situación actual 3
2.2 Lo que pide la gerencia 4
2.3 Alcance del prototipo 4
3. Análisis de requisitos 4
3.1 Técnicas empleadas 4
3.2 Requisitos funcionales 5
3.3 Requisitos no funcionales como escenarios de calidad 5
4. Arquitectura propuesta 6
4.1 Estilo arquitectónico 6
4.2 Componentes y responsabilidades 7
4.3 Relación con los principios de la unidad 7
4.4 Decisiones y compromisos 7
5. Diseño de la arquitectura mediante UML 8
5.1 Diagrama de casos de uso 8
5.2 Diagrama de clases 9
5.3 Diagrama de secuencia 10
5.4 Diagrama de despliegue 11
6. Selección y justificación de patrones de diseño 11
6.1 Justificación de los patrones centrales 12
7. Prototipo funcional en la nube 13
7.1 Tecnologías 13
7.2 Despliegue 13
7.3 Funcionalidad 14
7.4 Evidencia de los patrones 15
7.5 Cumplimiento de los escenarios de calidad 17
8. Conclusiones 17
Trabajo futuro 17
9. Referencias 18

1. Introducción

La arquitectura de software es la organización fundamental de un sistema, expresada en sus componentes, las relaciones entre ellos y con su entorno, y los principios que guían su diseño y evolución (ISO/IEC/IEEE, 2011; Bass et al., 2012). Las decisiones arquitectónicas se toman temprano y son costosas de revertir, por lo que deben responder a los atributos de calidad que necesita el negocio y no solo a la funcionalidad.

Este informe presenta la solución que el equipo propone para CapacitaPro S.A.S., una empresa ficticia que ofrece capacitación virtual obligatoria a otras empresas y cuya infraestructura actual, una instalación de WordPress por cliente, ya no soporta su crecimiento. La propuesta, llamada **AulaNube**, es una plataforma multiempresa construida con **microservicios**, un **API Gateway** y **comunicación asíncrona por eventos**, desplegada en Oracle Cloud Infrastructure (OCI) dentro de la capa gratuita.

El documento sigue el recorrido del proyecto: el caso y sus problemas (sección 2), los requisitos funcionales y no funcionales obtenidos con técnicas de análisis (sección 3), el estilo arquitectónico y su relación con los principios de modularidad de la unidad (sección 4), los diagramas UML de casos de uso, clases, secuencia y despliegue (sección 5), los patrones de diseño para la nube y su justificación (sección 6) y el prototipo funcional, con las pruebas que demuestran cada patrón (sección 7).

> **Objetivo.** Diseñar e implementar una arquitectura modular y escalable que permita a CapacitaPro atender a todos sus clientes desde una sola plataforma, absorber los picos de los cierres de campaña sin degradar el servicio, aislar fallos entre componentes y operar a bajo costo en la nube.

2. Caso de estudio: CapacitaPro S.A.S.

CapacitaPro S.A.S. es una empresa de Bucaramanga, fundada en 2017, que diseña y administra cursos virtuales que la ley o los contratos hacen obligatorios: inducción al Sistema de Gestión de Seguridad y Salud en el Trabajo (SG-SST), trabajo seguro en alturas, protección de datos personales según la Ley 1581 de 2012 y atención humanizada en salud. Atiende a 38 empresas cliente, desde constructoras de 80 trabajadores hasta una red de clínicas de 2.600 colaboradores, con unos 14.000 usuarios activos.

2.1 Situación actual

Cada cliente recibe su propia instalación de WordPress con un plugin de LMS en un servidor virtual alquilado; hoy hay 38 instalaciones en cinco servidores. El modelo funcionó con los primeros clientes, pero presenta los problemas de la tabla 1.

Tabla 1. Problemas identificados en la operación actual

Problema	Evidencia
Picos de carga en los cierres de campaña	El cierre de la campaña de alturas de una constructora (1.100 trabajadores) dejó el servidor sin respuesta 40 minutos y afectó a otros siete clientes alojados en la misma máquina.
El certificado se genera dentro de la solicitud	Respuestas de 8 a 25 s al terminar un curso en horas pico; si el usuario cierra la página, el certificado no se genera.

Problema	Evidencia
El correo de aviso bloquea la operación	Doce reclamos en un trimestre por cursos terminados que no quedaron registrados porque el servidor de correo tardó.
Mantenimiento multiplicado	Cada actualización se aplica 38 veces; dos personas dedican cerca del 40 % de su tiempo a ello.
Sin verificación de certificados	Auditores y contratantes deben pedir confirmación por correo; tarda días.

2.2 Lo que pide la gerencia

1. Una sola plataforma multiempresa en la nube, donde cada cliente vea solo su información.
2. Que el cierre de campaña de un cliente no afecte a los demás ni haga esperar al usuario.
3. Certificados con código verificable por terceros, sin intervención del equipo.
4. Que la caída de una parte (certificados, correo) no detenga el resto de la plataforma.
5. Crecer por partes: agregar capacidad solo donde hace falta.
6. Costo de infraestructura bajo durante el piloto.
7. Cumplir la Ley 1581 de 2012: datos aislados por cliente, contraseñas cifradas y acceso restringido.

2.3 Alcance del prototipo

El prototipo cubre el flujo central: ingreso, catálogo por empresa, avance de lecciones, finalización, emisión asíncrona del certificado, avisos, verificación pública, gestión básica de la empresa y un panel para observar la arquitectura en funcionamiento. Quedan fuera los pagos, la facturación, el contenido multimedia y la integración con un proveedor de correo real.

3. Análisis de requisitos

3.1 Técnicas empleadas

Tabla 2. Técnicas de análisis de requisitos

Técnica	Aplicación en el proyecto
Análisis documental	Lectura del caso y de los incidentes reportados para extraer necesidades y restricciones.
Entrevista simulada	El equipo representó a la gerencia, a una administradora de empresa cliente y a un colaborador que toma un curso; de cada papel salieron necesidades y prioridades.
Historias de usuario	Cada requisito funcional se redactó como «Como… quiero… para…» y se trazó a un caso de uso.
Escenarios de atributos de calidad	Los requisitos no funcionales se expresaron como escenarios medibles (estímulo, entorno, respuesta y medida), según Bass et al. (2012).
Priorización MoSCoW	Debe, Debería, Podría. Lo marcado como «Debe» entra al prototipo.

3.2 Requisitos funcionales

Tabla 3. Requisitos funcionales

ID	Historia de usuario	Prioridad	CU
RF01	Como usuario quiero iniciar sesión con correo y clave para acceder a la plataforma de mi empresa.	Debe	CU01
RF02	Como estudiante quiero ver solo los cursos de mi empresa, con mi avance.	Debe	CU02
RF03	Como estudiante quiero inscribirme en un curso para que se registre mi avance.	Debe	CU03
RF04	Como estudiante quiero marcar lecciones como vistas y ver mi porcentaje de avance.	Debe	CU04
RF05	Como estudiante quiero que al ver la última lección el curso quede completado.	Debe	CU05
RF06	Como estudiante quiero un certificado PDF con código único al completar un curso.	Debe	CU06, CU07
RF07	Como estudiante quiero un aviso al completar el curso y cuando mi certificado esté listo.	Debe	CU08
RF08	Como verificador externo quiero consultar un certificado por su código, sin cuenta.	Debe	CU09
RF09	Como administrador quiero crear cursos con sus lecciones para mi empresa.	Debe	CU10
RF10	Como administrador quiero registrar colaboradores de mi empresa.	Debería	CU11
RF11	Como administrador quiero ver inscritos, completados y certificados de mi empresa.	Debe	CU12
RF12	Como administrador quiero cerrar una campaña masiva sin que la plataforma se degrade.	Debe	CU13
RF13	Como equipo técnico quiero ver en vivo la salud de cada servicio, sus circuitos y la cola.	Debería	CU14
RF14	Como administrador quiero cargar colaboradores desde un archivo.	Podría	—

3.3 Requisitos no funcionales como escenarios de calidad

Tabla 4. Requisitos no funcionales expresados como escenarios de calidad

ID	Atributo	Escenario	Medida de aceptación
RNF01	Escalabilidad	Un cliente cierra una campaña: 150 colaboradores terminan el curso en el mismo minuto.	La finalización responde en menos de 1 s y todos los certificados se emiten sin errores.
RNF02	Rendimiento	Un estudiante completa la última lección en operación normal.	Respuesta < 500 ms; certificado disponible en < 10 s.
RNF03	Disponibilidad	El servicio de certificados se cae.	Cursos y avisos siguen respondiendo; tras 3 fallos el gateway responde en < 300 ms sin esperar al servicio caído.
RNF04	Confiabilidad	Redis o el trabajador no están disponibles cuando un curso se completa.	Ningún certificado se pierde y un evento repetido no genera duplicados.
RNF05	Seguridad y privacidad	Un usuario de la empresa A intenta leer datos de la empresa B o falsificar su rol.	Respuesta 403/404; claves con bcrypt; servicios internos inalcanzables desde fuera.
RNF06	Equidad entre inquilinos	Un cliente genera ráfagas de solicitudes.	Sobre 20 solicitudes/s (ráfaga de 40) recibe 429; los demás clientes no se afectan.

ID	Atributo	Escenario	Medida de aceptación
RNF07	Modificabilidad	Se cambia el envío de avisos por un correo real.	El cambio toca solo el servicio de notificaciones.
RNF08	Desplegabilidad	Se publica una versión nueva de un servicio.	Se despliega solo ese servicio, sin detener los demás.
RNF09	Costo	Operación del piloto.	Infraestructura dentro de la capa Always Free de OCI.
RNF10	Observabilidad	Una solicitud falla en un servicio interno.	Se sigue con el identificador de correlación y el panel de estado.

4. Arquitectura propuesta

4.1 Estilo arquitectónico

Se eligió una arquitectura de **microservicios con comunicación híbrida**: REST síncrono para lo que el usuario consulta y espera ver de inmediato, y **eventos asíncronos** para lo que puede ocurrir después (emitir el certificado y avisar). Los escenarios RNF01, RNF03 y RNF08 exigen escalar y aislar fallos *por partes*: en un cierre de campaña la emisión de certificados necesita varias réplicas, mientras que identidad no. Un monolito modular habría sido más sencillo de operar, pero escala y falla como una sola unidad, que es justamente el problema que sufre CapacitaPro (Red Hat, 2023; Redwerk, 2025a). Una arquitectura solo de eventos se descartó porque las consultas del estudiante necesitan respuesta inmediata.

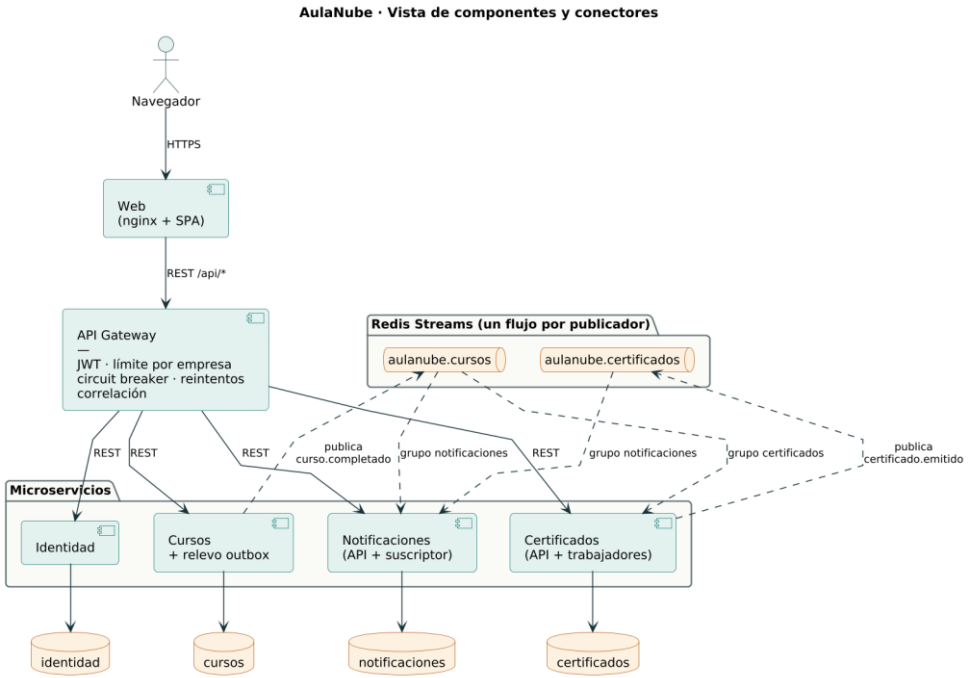


Figura 1. Vista de componentes y conectores. Las flechas continuas son llamadas REST; las punteadas, eventos en Redis Streams.

4.2 Componentes y responsabilidades

Tabla 5. Componentes de AulaNube

Componente	Responsabilidad	Datos propios
Web	Interfaz de una sola página; envía todo `/api` al gateway.	Ninguno
API Gateway	Única entrada: valida el JWT, reenvía la identidad, limita solicitudes por empresa, circuit breaker y reintentos.	Ninguno (estado en memoria)
Identidad	Empresas (inquilinos), usuarios, ingreso y emisión de tokens.	Base `identidad`
Cursos	Catálogo por empresa, inscripciones y progreso; publica `curso.completado`.	Base `cursos`
Certificados	Trabajadores que generan el PDF y publican `certificado.emitido`; verificación pública.	Base `certificados`
Notificaciones	Avisos en la plataforma y correo simulado.	Base `notificaciones`

4.3 Relación con los principios de la unidad

- **Alta cohesión.** Cada servicio agrupa un solo concepto del dominio, siguiendo la heurística de asignar un módulo a cada subproblema (Bass et al., 2012). Ningún servicio mezcla, por ejemplo, el cálculo del progreso con la generación de PDF.
- **Bajo acoplamiento.** Los servicios no se llaman entre sí. Cursos no sabe que certificados existe: solo publica un evento con un contrato pequeño (`tipo`, `evento_id`, `datos`). Agregar un nuevo interesado, como facturación, no exige tocar cursos.
- **Ocultamiento de información** (Parnas, 1972). Cada base de datos es un detalle interno de su servicio; cambiar dónde se guarda un PDF (de la base a Object Storage) no afecta a ningún otro módulo mientras la API y los eventos se mantengan.
- **Separación de responsabilidades.** Los asuntos transversales (autenticación, límites, resiliencia) viven en el gateway; los servicios solo resuelven su dominio.

4.4 Decisiones y compromisos

Tabla 6. Decisiones arquitectónicas y sus compromisos

Decisión	Beneficio	Costo aceptado
Certificados y avisos asíncronos	La respuesta al estudiante no depende del PDF ni del correo; los picos se absorben en la cola.	Consistencia eventual: el certificado aparece segundos después.
Una base por servicio en una sola instancia de PostgreSQL	Aislamiento lógico y permisos separados sin el costo de cuatro servidores.	La instancia es un punto compartido; en producción se separaría o se usaría un servicio administrado.
Redis Streams en lugar de un bróker dedicado	Grupos de consumidores, confirmación y reclamo de pendientes con muy poca memoria.	Menos garantías de durabilidad que Kafka; se mitiga con AOF y la bandeja de salida.
Código común copiado en cada servicio	Ningún servicio depende del ciclo de versiones de una librería compartida.	Duplicación de unas 150 líneas.

5. Diseño de la arquitectura mediante UML

Los diagramas se elaboraron en PlantUML siguiendo UML 2.5.1 (Object Management Group, 2017). El código fuente de cada uno está en `docs/uml/` del repositorio, de modo que evolucionan junto con el código.

5.1 Diagrama de casos de uso

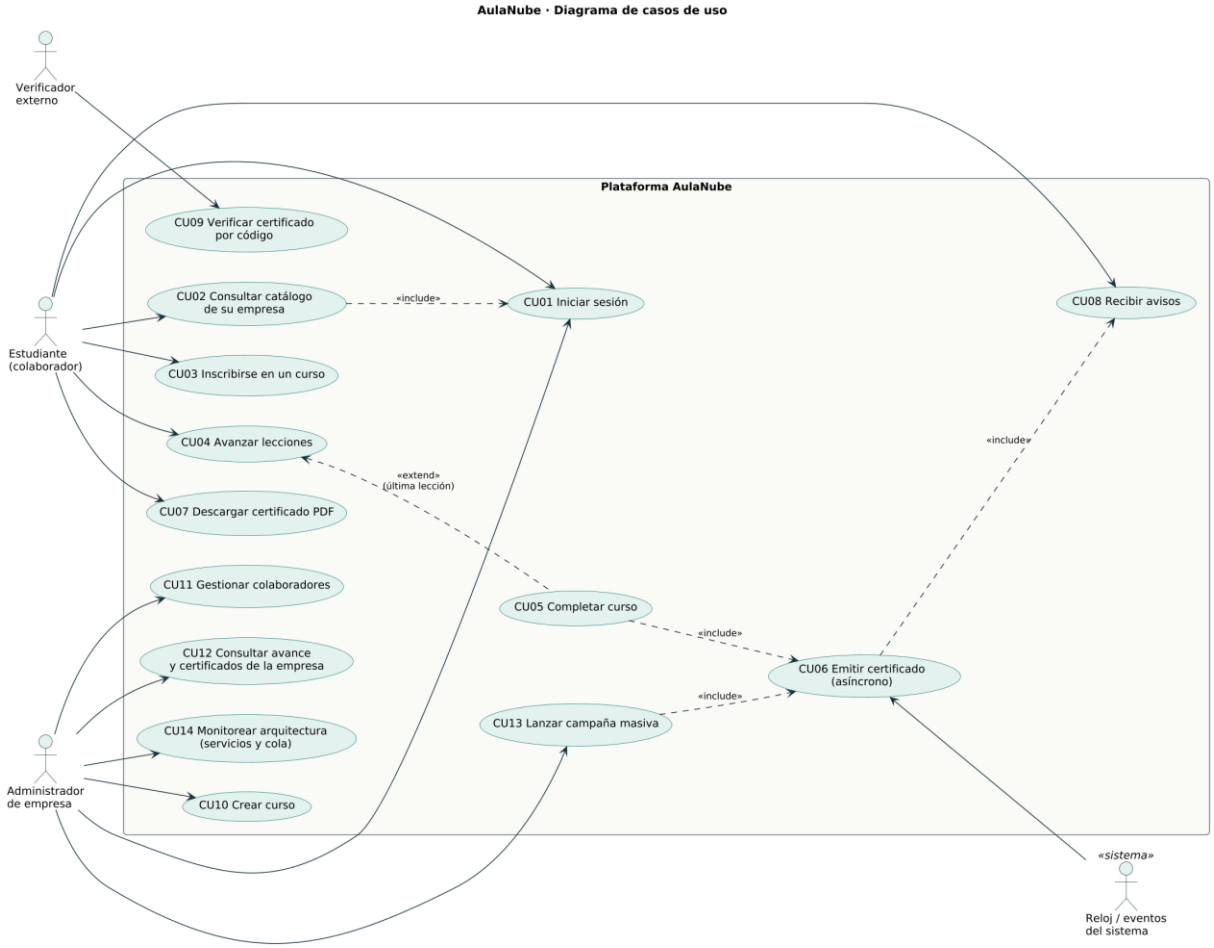


Figura 2. Casos de uso de AulaNube.

Descripción. Cuatro actores interactúan con la plataforma: el estudiante (colaborador de una empresa cliente), el administrador de la empresa, el verificador externo y el propio sistema, que dispara la emisión del certificado. CU05 «Completar curso» extiende el avance de lecciones cuando se marca la última e incluye CU06 «Emitir certificado», que a su vez incluye CU08 «Recibir avisos». La campaña masiva (CU13) también desencadena la emisión.

Relevancia arquitectónica. El diagrama muestra que CU06 no lo inicia una persona sino un evento: es la primera señal de que la emisión debe ser asíncrona. También muestra que el verificador externo no tiene cuenta, lo que obliga a una ruta pública controlada en el gateway.

5.2 Diagrama de clases

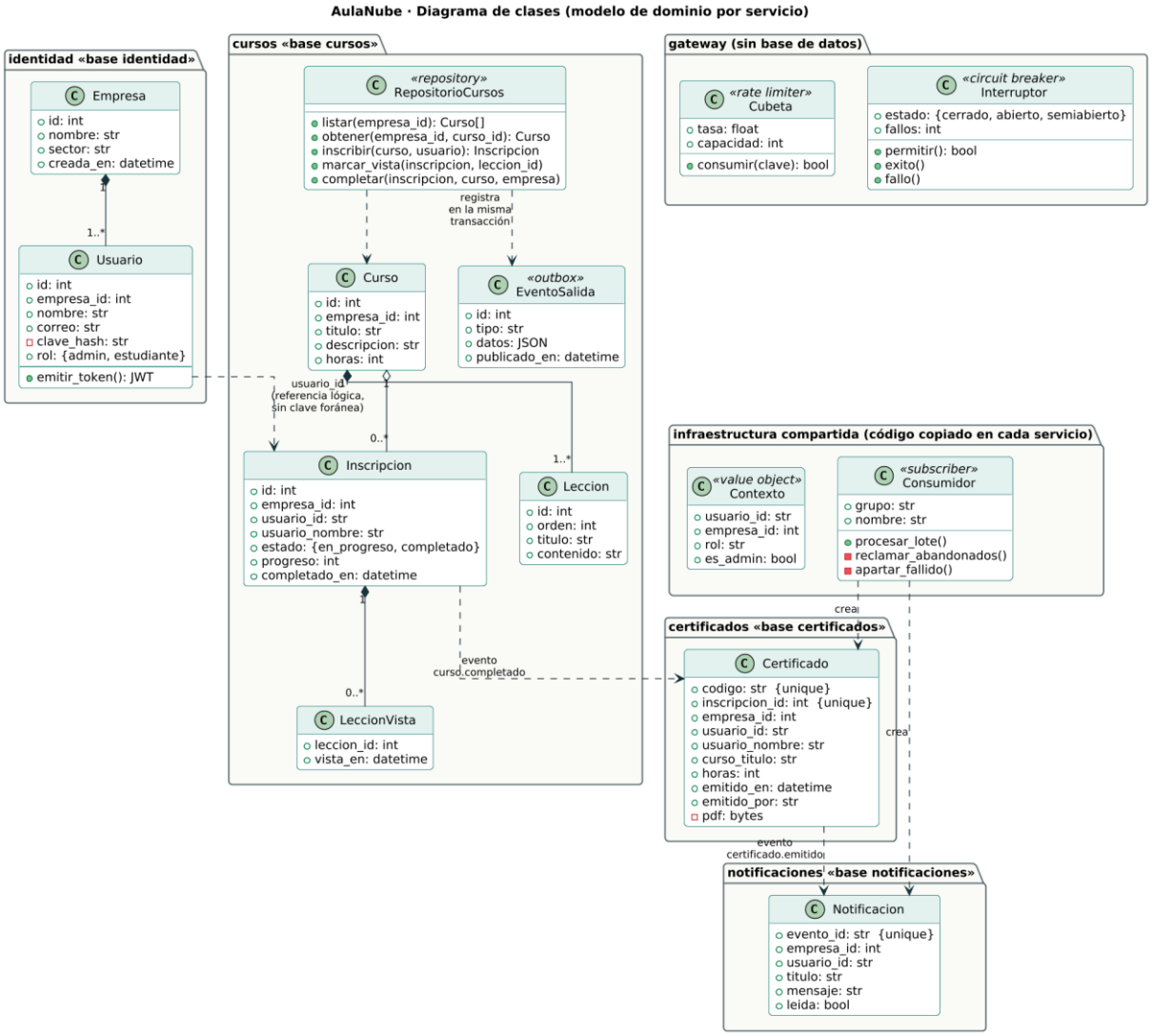


Figura 3. Modelo de dominio por servicio. Cada paquete corresponde a un servicio y a su base de datos.

Descripción. Cada paquete agrupa las clases de un servicio. `Empresa` y `Usuario` viven en identidad; `Curso`, `Leccion`, `Inscripcion`, `LeccionVista` y `EventoSalida` en cursos; `Certificado` y `Notificacion` en sus servicios. Las relaciones entre paquetes son *referencias lógicas* (`usuario_id`, `inscripcion_id`) o eventos, nunca claves foráneas entre bases. `RepositorioCursos` encapsula el acceso a datos e impone el filtro por empresa; `Interruptor` y `Cubeta` modelan el circuit breaker y el límite de solicitudes del gateway.

Relevancia arquitectónica. El diagrama comprueba la decisión *Database per Service*: ninguna asociación cruza un paquete. Las restricciones únicas de `Certificado.inscripcion_id` y `Notificacion.evento_id` son las que hacen idempotentes a los consumidores.

5.3 Diagrama de secuencia

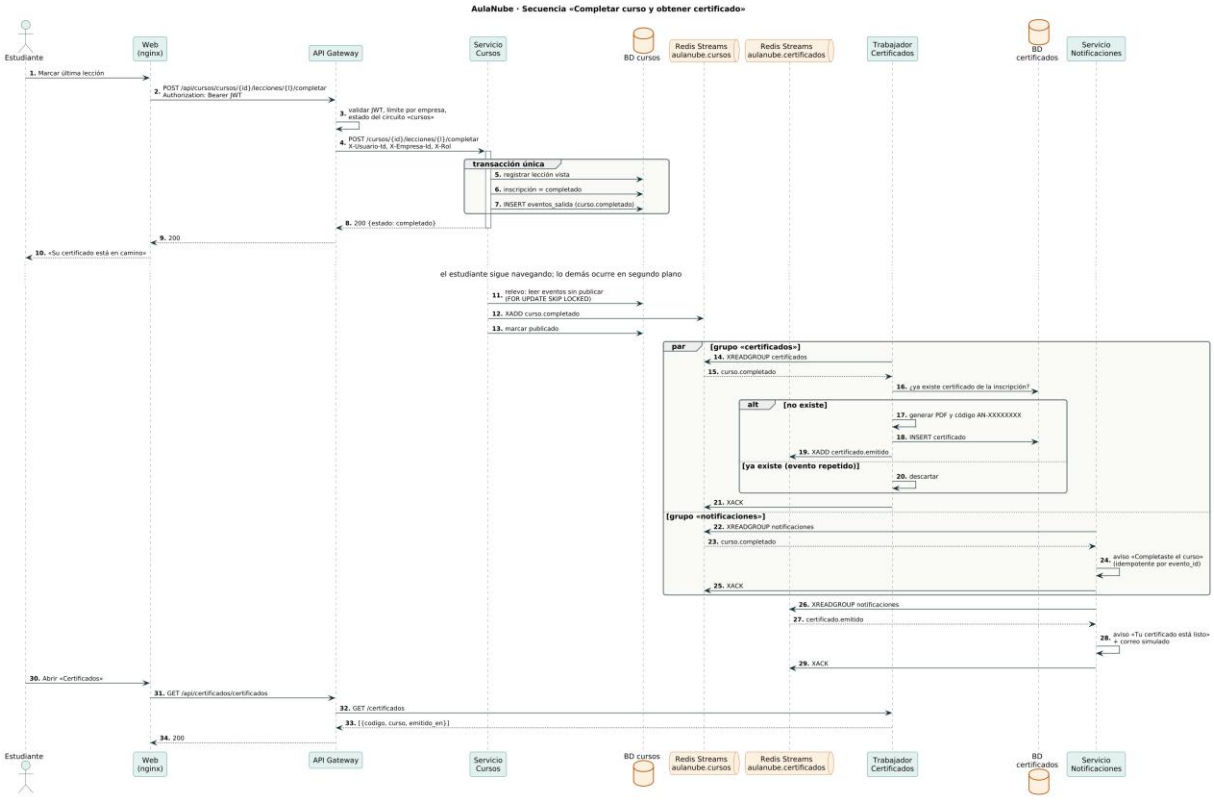


Figura 4. Secuencia «Completar curso y obtener certificado».

Descripción. El estudiante marca la última lección (pasos 1 a 4). Cursos registra la lección, cambia la inscripción a completada y guarda el evento en `eventos_salida` **en una sola transacción** (5 a 7), y responde de inmediato (8 a 10). En segundo plano, el relevo publica el evento en `aulanube.cursos` (11 a 13). Los grupos *certificados* y *notificaciones* lo reciben en paralelo; el trabajador verifica que no exista ya un certificado, genera el PDF, lo guarda y publica `certificado.emitido` en `aulanube.certificados`, que notificaciones convierte en el aviso «Tu certificado está listo».

Relevancia arquitectónica. Es el escenario que resuelve los problemas del caso: el usuario no espera por el PDF ni por el correo (RNF02), el pico de una campaña queda en la cola (RNF01) y, si Redis falla, el evento sigue en la base hasta poder publicarse (RNF04).

5.4 Diagrama de despliegue

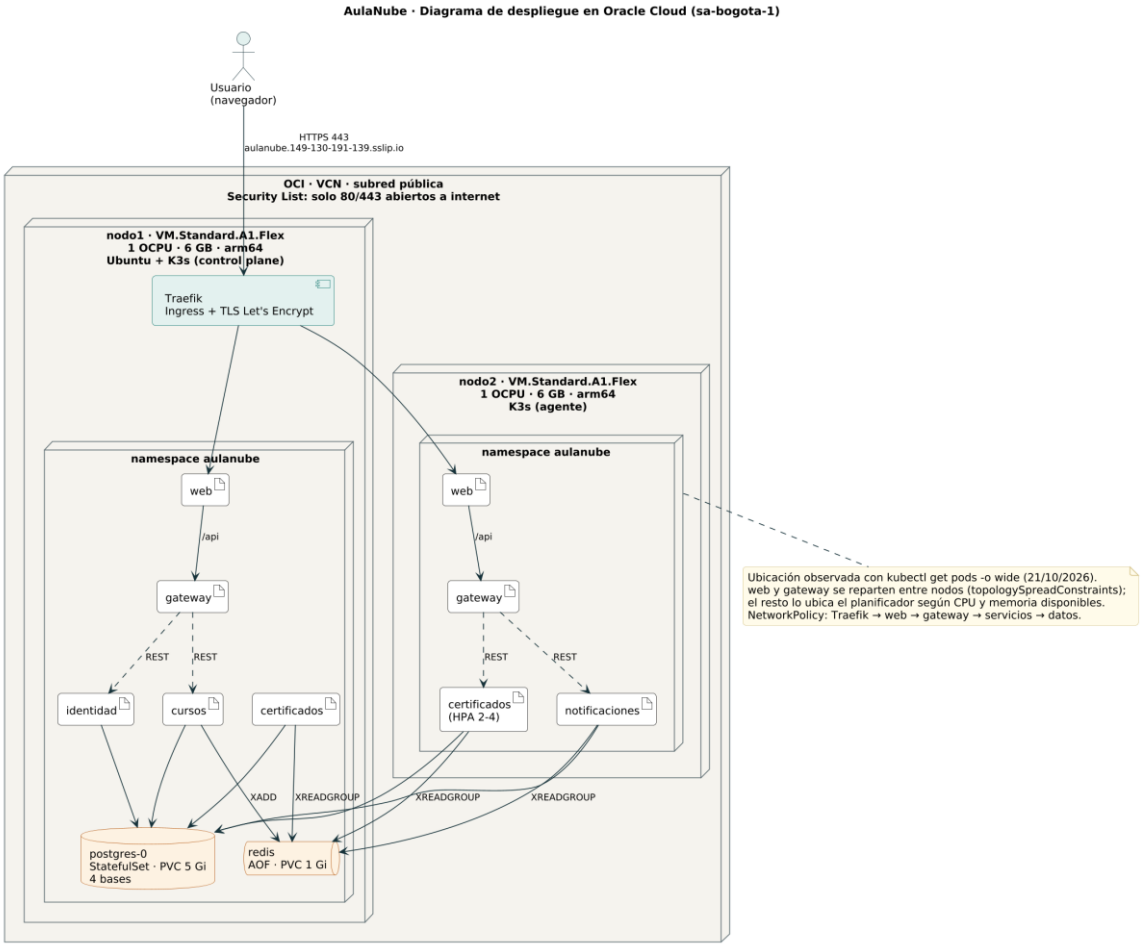


Figura 5. Despliegue en OCI (región sa-bogota-1) sobre dos nodos Ampere A1 con K3s.

Descripción. Dos instancias `VM.Standard.A1.Flex` de 1 OCPU y 6 GB (arm64) forman un clúster K3s. Traefik, incluido en K3s, termina TLS con un certificado de Let's Encrypt para `aulanube.149-130-191-139.sslip.io`. La web y el gateway tienen una réplica en cada nodo; certificados tiene de 2 a 4 réplicas según CPU (HPA). PostgreSQL corre como StatefulSet con volumen persistente y Redis con persistencia AOF. Las políticas de red solo permiten el camino Traefik → web → gateway → servicios → datos.

Relevancia arquitectónica. Muestra cómo se cumplen disponibilidad (réplicas en nodos distintos), escalabilidad (HPA), seguridad (solo 80/443 públicos y segmentación interna) y costo (todo dentro de la franquicia Always Free de Ampere A1).

6. Selección y justificación de patrones de diseño

Los patrones se seleccionaron a partir de los escenarios de calidad y del catálogo de patrones de diseño para la nube (Microsoft, s. f.; Richardson, 2018). Blas et al. (2019) señalan que estos patrones deben modelarse y verificarse como parte de la arquitectura y no como detalles de implementación; por eso cada uno aparece en los diagramas de la sección 5 y se prueba en la sección 7.

Tabla 7. Patrones seleccionados y su justificación

Patrón	Problema del caso	Aplicación en AulaNube	RNF
API Gateway y **Gateway Offloading**	El navegador no debe conocer los servicios internos y cada servicio repetiría la validación del token y los límites.	Un solo punto de entrada `/api/{servicio}/…`. Valida el JWT, borra las cabeceras de identidad que envíe el cliente y reenvía `X-Usuario-Id`, `X-Empresa-Id` y `X-Rol`; agrega un identificador de correlación.	05, 10
Database per Service	Con una base compartida, un cambio de esquema de cursos podría romper certificados.	Una base y un usuario por servicio; cada usuario solo puede conectarse a la suya.	07, 08
Publisher/Subscriber	Cursos no debe saber quién reacciona cuando un curso termina.	Redis Streams con un flujo por publicador y un grupo de consumidores por servicio suscrito.	07
Transactional Outbox	Si Redis falla justo después de guardar el curso, el evento se perdería.	Tabla `eventos_salida` escrita en la misma transacción; un hilo la publica con `FOR UPDATE SKIP LOCKED`. Entrega «al menos una vez».	04
Queue-Based Load Leveling	El cierre de campaña tumbaba el servidor.	`POST /campanas` responde 202 al instante; la cola absorbe el pico y los trabajadores la procesan a su ritmo.	01
Competing Consumers	Un solo trabajador tardaría demasiado con cientos de certificados.	Varias réplicas en el mismo grupo; Redis entrega cada evento a una sola. HPA de 2 a 4 réplicas. Los eventos de una réplica caída se reclaman con `XAUTOCLAIM`.	01, 04
Circuit Breaker y **Retry**	Un servicio caído hacía esperar cada solicitud y arrastraba al resto.	Tras 3 fallos seguidos el circuito se abre 15 s y responde de inmediato; luego deja pasar una prueba (semiabierto). Las lecturas GET se reintentan una vez.	03
Rate Limiting (cubeta de fichas)	Un cliente no debe acaparar la plataforma.	20 solicitudes/s con ráfaga de 40 por empresa; responde 429 al excederse.	06
Health Endpoint Monitoring	Kubernetes y el equipo deben saber si cada servicio está vivo.	`/salud` en cada servicio, sondas *readiness* y *liveness*, panel `/api/estado`.	03, 10
Dead Letter	Un evento malformado no debe bloquear la cola.	Tras 3 intentos fallidos el evento se aparta en `aulanube.fallidos`.	04
External Configuration Store	Credenciales y direcciones cambian entre local y nube.	Variables de entorno desde Secrets de Kubernetes o `.env`; ningún secreto en el repositorio.	05, 08
Repository	El filtro por empresa no debe depender de que cada consulta lo recuerde.	`RepositorioCursos` encapsula las consultas y aplica el filtro de inquilino en un solo lugar (Fowler, 2002).	05

6.1 Justificación de los patrones centrales

Circuit Breaker. Nygard (2018) describe cómo una integración lenta consume hilos y conexiones hasta tumbar al llamador: es exactamente el caso del correo que bloqueaba la finalización del curso. El interruptor convierte un fallo lento en uno rápido y da tiempo al servicio para recuperarse. Cada réplica del gateway mantiene su propio interruptor; con dos réplicas, cada una necesita sus tres fallos para abrirse, lo que es aceptable a esta escala.

Outbox y consumidores idempotentes. Guardar en la base y publicar en la cola son dos operaciones que no pueden ser atómicas entre sí. La bandeja de salida las convierte en una sola

escritura local y traslada el problema a «publicar al menos una vez», que se resuelve haciendo idempotentes a los consumidores con restricciones únicas (Richardson, 2018).

Load Leveling con Competing Consumers. Separan la tasa a la que llegan las solicitudes de la tasa a la que se procesan. La capacidad de emisión se ajusta con el número de réplicas, sin cambiar código, y el resto de la plataforma no siente el pico (Microsoft, s. f.).

Patrones evaluados y descartados. *Saga* no se necesitó porque ningún flujo requiere deshacer pasos en varios servicios. *CQRS* se consideró para los informes del administrador, pero el volumen actual no lo justifica. *Service Mesh* resolvería reintentos y TLS interno, pero su consumo de recursos excede los nodos de 1 OCPU.

7. Prototipo funcional en la nube

7.1 Tecnologías

Tabla 8. Tecnologías del prototipo

Capa	Tecnología
Servicios	Python 3.12, FastAPI, SQLAlchemy 2, PyJWT, bcrypt, ReportLab, httpx
Datos y mensajería	PostgreSQL 16 (una base por servicio), Redis 7.4 Streams con AOF
Interfaz	HTML, CSS y JavaScript sin dependencias, servidos por nginx sin privilegios
Contenedores y orquestación	Docker, Docker Compose (local), K3s con Traefik, HPA y NetworkPolicy (nube)
Nube	Oracle Cloud Infrastructure, región sa-bogota-1, 2 × VM.Standard.A1.Flex (Always Free)
Pruebas	pytest con SQLite en memoria y Redis simulado (28 pruebas); prueba de humo de punta a punta

Repositorio: https://github.com/augustocco/aulanube · **Aplicación:** https://aulanube.149-130-191-139.sslip.io (cuentas de demostración en el README, clave `AulaNube-2026`).

7.2 Despliegue

Las imágenes se construyen de forma nativa en arm64 sobre el nodo 1, se importan en containerd de ambos nodos y se aplican cinco manifiestos: namespace y secretos, datos, servicios, web e ingreso, y políticas de red. El script `scripts/desplegar.sh` automatiza el proceso y permite desplegar un solo servicio sin detener los demás (RNF08). El estado observado del clúster es:

```
$ kubectl -n aulanube get pods -o wide
NAME                              READY   STATUS    NODE
certificados-7fc8f85b94-c7mw2     1/1     Running   tiendaandina-k8s-nodo2
certificados-7fc8f85b94-t9kk7     1/1     Running   tiendaandina-k8s-nodo1
cursos-876b88d5-hwwfj             1/1     Running   tiendaandina-k8s-nodo1
gateway-6ffbdbd846-lbz56          1/1     Running   tiendaandina-k8s-nodo2
gateway-6ffbdbd846-xdzwg          1/1     Running   tiendaandina-k8s-nodo1
identidad-cffdbd87f-ld4h8         1/1     Running   tiendaandina-k8s-nodo1
notificaciones-5cd6cfb4ff-5t652   1/1     Running   tiendaandina-k8s-nodo2
postgres-0                        1/1     Running   tiendaandina-k8s-nodo1
redis-5dfcbb9d87-bw6kd            1/1     Running   tiendaandina-k8s-nodo1
web-7d76dcf555-2r7pm              1/1     Running   tiendaandina-k8s-nodo1
web-7d76dcf555-qkfb7              1/1     Running   tiendaandina-k8s-nodo2

$ openssl s_client -connect aulanube.149-130-191-139.sslip.io:443 | openssl x509 -noout -issuer
issuer=C = US, O = Let's Encrypt, CN = YR2
```

7.3 Funcionalidad

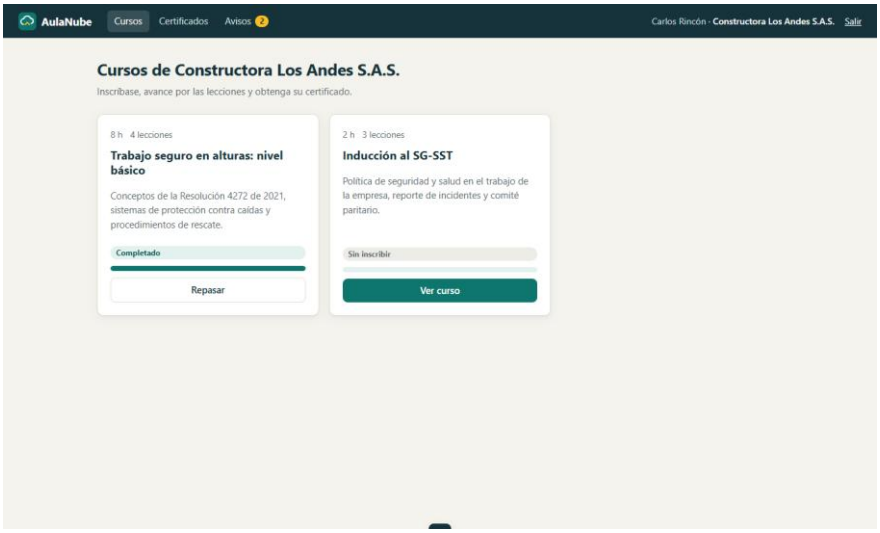


Figura 6. Catálogo de la empresa con el avance del estudiante.

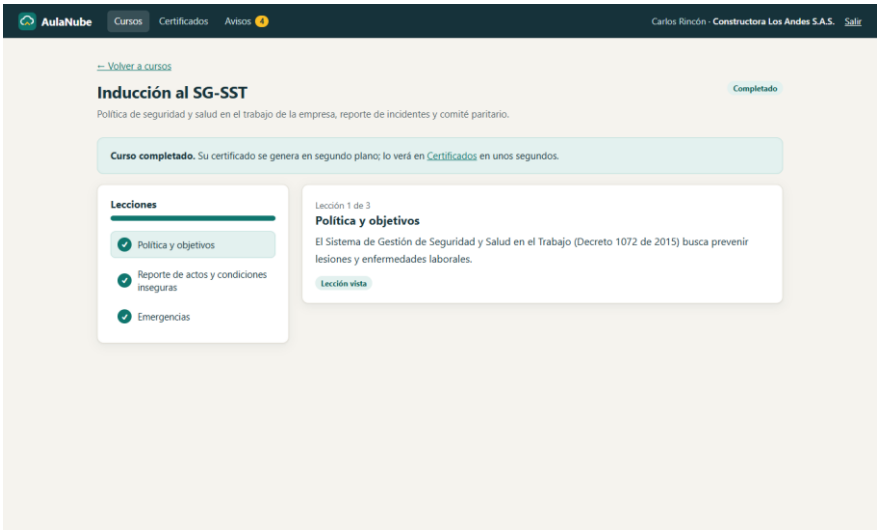


Figura 7. Curso completado: el certificado se genera en segundo plano.

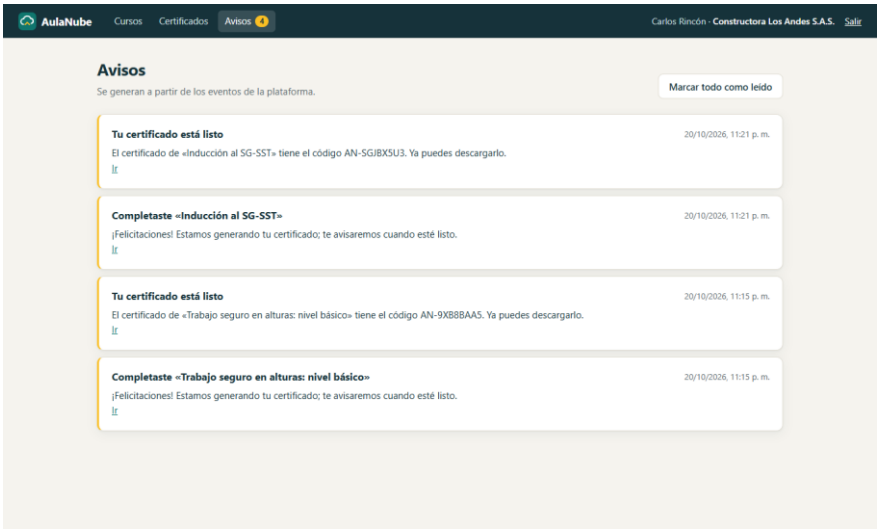


Figura 8. Avisos generados por los eventos `curso.completado` y `certificado.emitido`.


AULANUBE · CAPACITAPRO S.A.S.

Certificado de finalización

Se certifica que

Carlos Rincón

colaborador(a) de Constructora Los Andes S.A.S., completó el curso

Trabajo seguro en alturas: nivel básico

con una intensidad de 8 horas, el 21 de octubre de 2026.

Dirección académica · CapacitaPro

Código de verificación: AN-XA7TAUCS

Verifique la autenticidad en https://aulanube.149-130-191-139.sslip.io/#/verificar/AN-XA7TAUCS

Figura 9. Certificado PDF emitido por un trabajador, con código verificable públicamente.

7.4 Evidencia de los patrones

Nivelación de carga y consumidores en competencia (RNF01)

Desde la gestión de la empresa se lanzó una campaña de 200 participantes. El servicio de cursos respondió **202 en menos de 300 ms** y el panel de arquitectura muestra la cola del grupo *certificados* bajando mientras dos réplicas la procesan, a unos 6,5 certificados por segundo; el resto de la plataforma siguió respondiendo con normalidad. En la tabla de certificados del administrador, la columna «Trabajador» muestra qué réplica emitió cada uno.

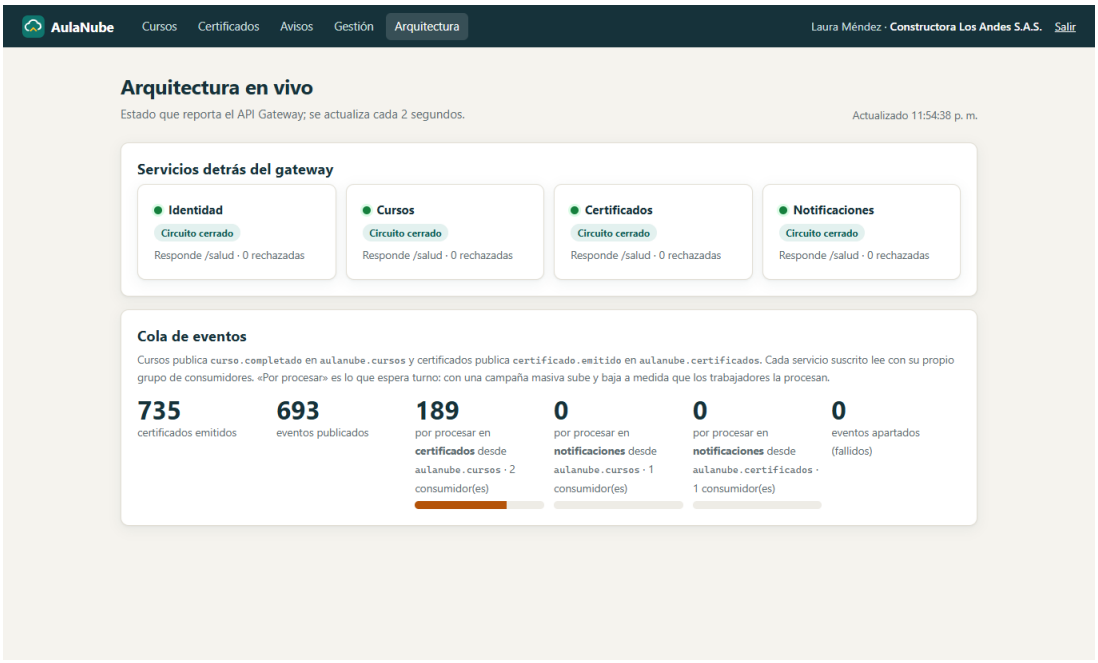


Figura 10. Panel de arquitectura durante una campaña de 200 participantes: 189 eventos por procesar en el grupo *certificados*, con dos consumidores.

Circuit breaker y aislamiento de fallos (RNF03)

Se escaló certificados a cero réplicas. Tras tres fallos, el gateway respondió con el circuito abierto, sin tocar el servicio caído, mientras el catálogo de cursos seguía respondiendo:

```
$ kubectl -n aulanube scale deploy certificados --replicas=0
intento 1: 503 en 0.112 s   X-Circuito: abierto
intento 2: 503 en 0.129 s   X-Circuito: abierto
...
cursos mientras tanto: 200 en 0.156 s
GET /api/estado -> certificados: {"estado": "abierto", "fallos_seguidos": 3,
"segundos_para_reintento": 6.2}
```

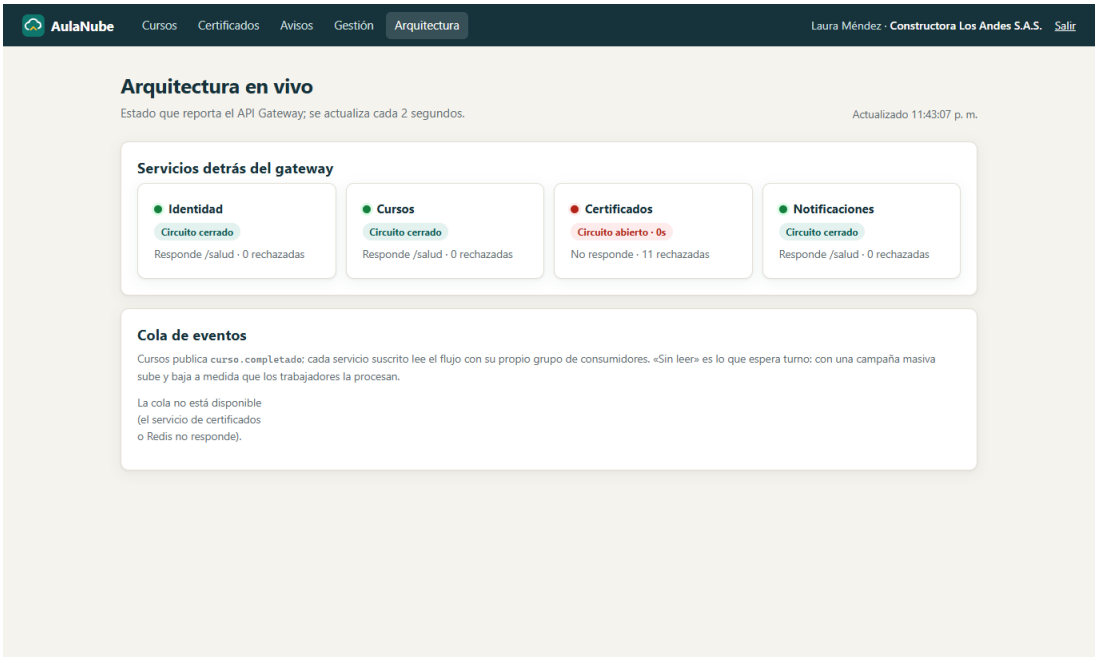


Figura 11. Circuito de certificados abierto; los demás servicios siguen disponibles.

Entrega garantizada (RNF04)

Con certificados apagado, una estudiante completó el curso «Inducción al SG-SST». El evento quedó en Redis (`lag 1` en el grupo *certificados*) y notificaciones ya había generado su aviso. Al volver a escalar el servicio a dos réplicas (23:43:23), el certificado quedó emitido a las 23:43:32, sin intervención manual.

Seguridad y segmentación (RNF05)

```
web -> identidad:8000      Connection refused   (solo el gateway llega a los servicios)
gateway -> postgres:5432   tiempo agotado        (solo los servicios llegan a los datos)
GET /api/cursos/cursos sin token -> 401
X-Rol: admin enviado por el cliente -> descartado; el servicio recibe el rol del JWT
```

Pruebas automatizadas

Las 28 pruebas unitarias y de integración pasan en x86 y en los nodos arm64 (gateway 10, identidad 6, cursos 5, certificados 5, notificaciones 2). Cubren el interruptor de circuito, el límite por empresa, el aislamiento entre inquilinos, la bandeja de salida, la idempotencia, el envío a fallidos y los permisos. La prueba de humo contra la nube recorre el flujo completo:

```
$ python scripts/prueba_humo.py https://aulanube.149-130-191-139.sslip.io
  ✔ sin token el gateway responde 401
  ✔ ingreso de administradora y de un colaborador creado para la prueba
  ✔ catálogo aislado por empresa (2 cursos de Los Andes)
  ✔ certificado AN-9S7FZRSY emitido por certificados-7fc8f85b94-c7mw2
  ✔ PDF descargado y verificación pública sin token
  ✔ aviso «Tu certificado está listo» recibido
  ✔ campaña de 40 participantes: respuesta en 195 ms, certificados listos en 7.4 s
  ✔ los cuatro servicios responden y sus circuitos están cerrados
```

7.5 Cumplimiento de los escenarios de calidad

Tabla 9. Resultado de los escenarios de calidad

RNF	Medida esperada	Resultado medido
RNF01	Finalización masiva < 1 s, sin errores	202 en 195–300 ms; 200 certificados emitidos sin fallidos.
RNF02	Respuesta < 500 ms; certificado < 10 s	Finalización ≈ 150 ms; certificado en 1–2 s en reposo.
RNF03	Respuesta < 300 ms con un servicio caído	110–150 ms con circuito abierto; cursos sin afectación.
RNF04	Ningún certificado perdido	Evento retenido y procesado al recuperar el servicio.
RNF05	Aislamiento y segmentación	Verificado con pruebas y con NetworkPolicy en el clúster.
RNF09	Capa gratuita	2 nodos de 1 OCPU y 6 GB: dentro de la franquicia Ampere A1.

Limitaciones conocidas: el límite de solicitudes y el estado de los circuitos viven en la memoria de cada réplica del gateway; PostgreSQL y Redis tienen una sola réplica; el correo es simulado. Son las primeras mejoras para pasar del piloto a producción.

8. Conclusiones

- Los problemas de CapacitaPro eran de **arquitectura**, no de código: un servidor por grupo de clientes y un flujo síncrono hacían que cualquier pico o fallo se propagara. Cambiar a una arquitectura modular resolvió esos síntomas sin optimizar una sola línea de lógica de negocio, lo que confirma que las decisiones estructurales tempranas determinan la calidad del sistema (Bass et al., 2012).
- Partir de **escenarios de calidad medibles** permitió elegir patrones con criterio y comprobarlos: cada patrón del prototipo responde a un RNF concreto y tiene una prueba o una medición que lo respalda.
- La **alta cohesión, el bajo acoplamiento y el ocultamiento de información** se hicieron concretos: una base por servicio, eventos con contratos pequeños y un gateway que concentra lo transversal. Gracias a ello, durante el proyecto se reorganizaron los flujos de eventos tocando solo dos servicios.
- La asincronía tiene un costo real: consistencia eventual, entrega «al menos una vez» y la necesidad de consumidores idempotentes. Hacer explícitos estos compromisos es parte del trabajo del arquitecto.
- Una arquitectura en la nube no exige un presupuesto alto: el prototipo completo corre en la capa gratuita de OCI con dos nodos ARM, con TLS, escalado automático y segmentación de red.

Trabajo futuro

Mover los PDF a OCI Object Storage, usar PostgreSQL administrado con réplica, compartir el estado del gateway en Redis, integrar OCI Email Delivery, agregar trazas distribuidas con OpenTelemetry y un pipeline de CI/CD que construya y despliegue cada servicio por separado.

9. Referencias

Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley.

Blas, M. J., Leone, H. P., & Gonnet, S. M. (2019). Modelado y verificación de patrones de diseño de arquitectura de software para entornos de computación en la nube. En *Proceedings del Congreso Nacional de Ingeniería Informática/Sistemas de Información*.

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial 48.587.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Garlan, D., & Shaw, M. (1993). *An introduction to software architecture*. Addison-Wesley.

ISO/IEC/IEEE. (2011). *ISO/IEC/IEEE 42010:2011 Systems and software engineering — Architecture description*. International Organization for Standardization.

Microsoft. (s. f.). *Cloud design patterns*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/patterns/

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.

Object Management Group. (2017). *OMG Unified Modeling Language (UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Oracle. (s. f.). *Always Free resources*. Oracle Cloud Infrastructure Documentation. https://docs.oracle.com/en-us/iaas/Content/FreeTier/freetier*topic-Always*Free_Resources.htm

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM, 15*(12), 1053–1058.

Red Hat. (2023). *¿Qué son los microservicios? Ventajas de la arquitectura de microservicios*. https://www.redhat.com/es/topics/microservices/what-are-microservices

Redis. (s. f.). *Redis Streams*. Redis Documentation. https://redis.io/docs/latest/develop/data-types/streams/

Redwerk. (2025a). *Arquitectura monolítica frente a la de microservicios para .NET*. https://redwerk.es/blog/arquitectura-monolitica-vs-microservicios/

Redwerk. (2025b). *10 principios clave para construir una arquitectura de software escalable y un crecimiento a largo plazo*. https://redwerk.es/blog/arquitectura-de-software-escalable/

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.

Taylor, R. N., Medvidovic, N., & Dashofy, E. M. (2010). *Software architecture: Foundations, theory, and practice*. Wiley.